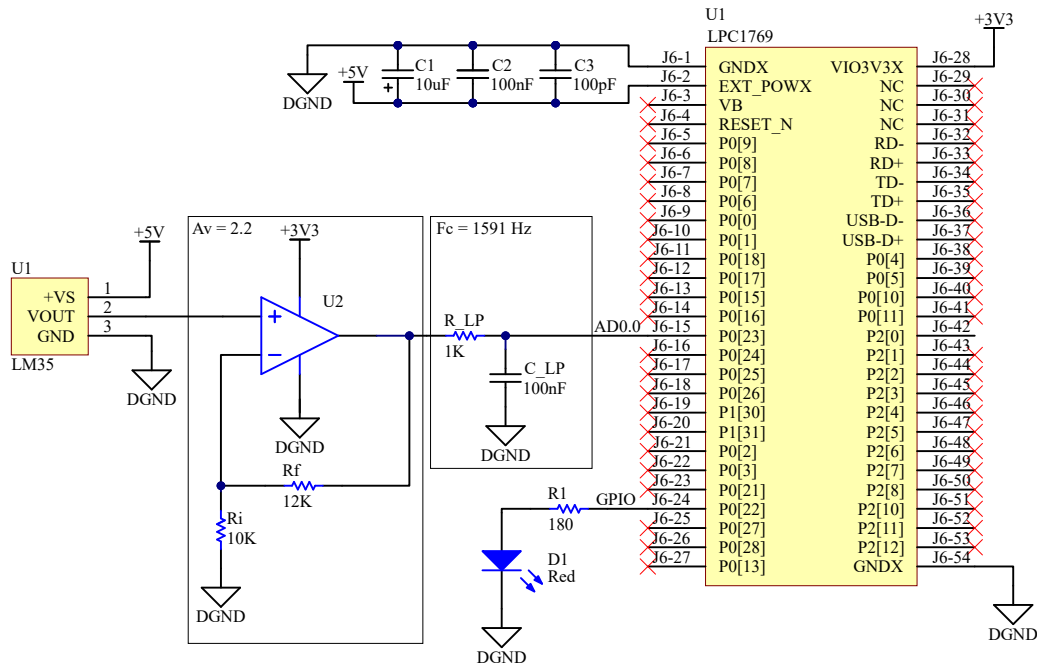


```

1  /*=====
2  * @file     Ejercicio_6_1_2.c
3  *
4  * @brief     Se requiere monitorizar la temperatura ambiente mediante un sensor
5  *            analógico LM35 conectado al canal AD0.0 del ADC..
6  *            Realizar el muestreo periódico de la señal y la detección de
7  *            sobrepaso de un umbral crítico de temperatura de 60 °C mediante una
8  *            alarma técnica visual de prioridad alta, conformada por un LED rojo
9  *            en el pin P0.22.
10 *
11 * @details   - Las señales de los sensores LM35 se acondicionan mediante
12 *             amplificadores operacionales configurados para maximizar el
13 *             aprovechamiento del rango dinámico del ADC.
14 *             - El acondicionamiento adapta la tensión generada por los
15 *             sensores al rango de entrada del LPC1769 (0 V a 3.3 V),
16 *             incrementando la resolución efectiva de la medición y
17 *             mejorando la inmunidad al ruido durante la adquisición.
18 *             - Base de tiempo de adquisición: 1 muestra por segundo (1 Hz)
19 *             mediante el disparo por hardware de MAT0.1 del Timer 0.
20 *             - Implementación en alto nivel mediante drivers (NXP LPC17xx).
21 *
22 * @author    Ing. E. Migliore
23 *
24 * @version   3.0
25  */

```



$$A_V = \frac{V_{out(max)}}{V_{in(max)}} = \frac{V_{out(max)}}{T_{LM35(max)} S_{LM35}} = \frac{3.3 \text{ V}}{150 \text{ }^{\circ}\text{C} * 10 \frac{\text{mV}}{^{\circ}\text{C}}} = 2.2$$

$$A_V = 1 + \frac{R_f}{R_i} \Rightarrow R_f = (A_V - 1)R_i = (2.2 - 1) * 10 \text{ k}\Omega = 12 \text{ k}\Omega$$

```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5  #include "lpc17xx_gpio.h"
6  #include "lpc17xx_pinsel.h"
7  #include "lpc17xx_timer.h"
8  #include "lpc17xx_adc.h"
9  #include <stdbool.h>
10
11 /*=====
12 * DEFINICIÓN DE CONSTANTES
13 *=====*/
14 /* Parámetros del ADC */
15 #define ADC_VREF_MV          3300U      // Tensión de Referencia ADC: 3.3 V
16 #define ADC_MAX_COUNTS      4095U      // Resolución de 12 bits (2^12 - 1)
17 #define ADC_SAMPLE_RATE_HZ  200000U    // ADC Conversion Rate = 200 kS/s
18
19 /* Parámetros de Adquisición */
20 #define PCLK_FREQ_HZ         25000000U  // PCLK = 25 MHz (Default @ CCLK = 100 MHz)
21 #define ADC_TRIGGER_RATE_HZ  1U          // FS = 1 Hz (1 muestra por segundo)
22 #define TMR0_PR_VALUE       1U          // TC incrementa cada 1 µs
23 #define US_PER_SEC          1000000U
24 #define TMR0_MR_VALUE \
25     ((US_PER_SEC / (2U * ADC_TRIGGER_RATE_HZ * TMR0_PR_VALUE)) - 1U)
26
27 /* Parámetros del Sensor LM35 */
28 #define LM35_SENSITIVITY_MV  10U        // Sensibilidad: 10 mV/°C
29
30 /* Canales ADC */
31 #define LM35_SENSOR_CHANNEL  ADC_CHANNEL_0
32
33 /* Pines de salida digital */
34 #define ALARM_HIGH_PORT      PINSEL_PORT_0
35 #define ALARM_HIGH_PIN      PINSEL_PIN_22
36 #define ALARM_HIGH_PIN_MASK (1U << 22)
37
38 /* Modalidades GPIO */
39 #define GPIO_INPUT           0U
40 #define GPIO_OUTPUT          1U
41
42 /* Umbrales de supervisión */
43 #define ALARM_TEMP_C         60U        // Umbral de alarma por sobrettemperatura: 60 °C
44
45 /*=====
46 * DEFINICIÓN DE VARIABLES GLOBALES
47 *=====*/
48 volatile uint16_t lm35_raw    = 0;
49 volatile uint32_t lm35_mv     = 0;
50 volatile uint32_t lm35_temp_c = 0;
51
52 /*=====
53 * PROTOTIPOS DE FUNCIONES
54 *=====*/
55 void cfgGPIO(void);
56 void cfgTMR0(uint32_t, uint32_t, uint8_t);
57 void cfgADC(uint32_t rate);
58
59 /*=====
60 * FUNCIÓN PRINCIPAL
61 *=====*/
62 int main(void) {
63
64     /*-----
65     * INICIALIZACIÓN DEL MCU
66     *-----*/
67     cfgGPIO();
68     cfgTMR0(TMR0_PR_VALUE, TMR0_MR_VALUE, TIM_EXTMATCH_TOGGLE);
69     cfgADC(ADC_SAMPLE_RATE_HZ);

```

```

70
71     /*-----*/
72     * INICIO PROGRAMA PRINCIPAL
73     *-----*/
74     while(1){
75
76         return 0;
77     }
78
79     /*=====
80     * DEFINICIONES DE FUNCIONES
81     *=====*/
82     /*****
83     * @brief   Configura las direcciones GPIO, modos de función y resistencias.
84     *
85     * @details - P0.22: Función GPIO como Salida Digital (Alarma de prioridad alta)
86     *           - P0.23: Función AD0.0 como Entrada Analógica (Sensor LM35)
87     *           - P1.29: Función MAT0.1 como Salida (Match Timer 0).
88     *
89     * @param   None
90     * @return  None
91     *****/
92     void cfgGPIO(void)
93     {
94         //Configuración del Pin P0.22
95         PINSEL_CFG_Type cfgPinAlarmHigh;
96         //Configuración del Pin P0.23
97         PINSEL_CFG_Type cfgPinSensorTemp;
98         //Configuración del Pin P1.29
99         PINSEL_CFG_Type cfgPinTriggerADC;
100
101         /* P0.22 como GPIO para alarma de prioridad alta */
102         //Selección de Puerto 0
103         cfgPinAlarmHigh.Portnum = PINSEL_PORT_0;
104         //Selección de Pin 22
105         cfgPinAlarmHigh.Pinnum = PINSEL_PIN_22;
106         //Funcionalidad de P0.22 como GPIO
107         cfgPinAlarmHigh.Funcnum = PINSEL_FUNC_0;
108
109         /* P0.23 como AD0.0 para el LM35 */
110         //Selección de Puerto 0
111         cfgPinSensorTemp.Portnum = PINSEL_PORT_0;
112         //Selección de Pin 23
113         cfgPinSensorTemp.Pinnum = PINSEL_PIN_23;
114         //Funcionalidad de P0.23 como AD0.0 (Canal 0)
115         cfgPinSensorTemp.Funcnum = PINSEL_FUNC_1;
116         //P0.23 Flotante
117         cfgPinSensorTemp.Pinmode = PINSEL_PINMODE_TRISTATE;
118         //Modalidad sin Drenador Abierto
119         cfgPinSensorTemp.OpenDrain = PINSEL_PINMODE_NORMAL;
120
121         /* P1.29 como MAT0.1 */
122         //Selección de Puerto 1
123         cfgPinTriggerADC.Portnum = PINSEL_PORT_1;
124         //Selección de Pin 29
125         cfgPinTriggerADC.Pinnum = PINSEL_PIN_29;
126         //Funcionalidad de P1.29 como MAT0.1
127         cfgPinTriggerADC.Funcnum = PINSEL_FUNC_3;
128         //Modalidad sin Drenador Abierto
129         cfgPinTriggerADC.OpenDrain = PINSEL_PINMODE_NORMAL;
130
131         //Inicialización de la alarma: P0.22
132         PINSEL_ConfigPin(&cfgPinAlarmHigh);
133         //Inicialización de sensor de temperatura: P0.23 (AD0.0)
134         PINSEL_ConfigPin(&cfgPinSensorTemp);
135         //Inicialización del trigger del ADC: P1.29 (MAT0.1)
136         PINSEL_ConfigPin(&cfgPinTriggerADC);
137
138         //Configuración y estado inicial del LED: P0.22

```

```

139     GPIO_SetDir(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK, GPIO_OUTPUT);
140     GPIO_ClearValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
141
142     return;
143 }
144
145 /*****
146  * @brief   Configura e inicializa el periférico TIMER0 en modo Timer.
147  *
148  * @details Genera una señal periódica en MAT0.1 destinada al disparo
149  *           por hardware del ADC.
150  *
151  *           La frecuencia de la señal generada depende de la base de tiempo
152  *           configurada para el Prescale Counter (PC), del valor de
153  *           coincidencia MR1 y de la modalidad de salida seleccionada para
154  *           MAT0.1.
155  *
156  * @param   <period_us> Tiempo de desborde del Prescale Counter (PC),
157  *                   expresado en microsegundos.
158  * @param   <match_value> Valor límite del registro de coincidencia MR1.
159  * @param   <mode> Modalidad de salida External Match Control (EMC1)
160  *                   aplicada sobre MAT0.1.
161  *
162  * @return  None.
163  *****/
164 void cfgTMR0(uint32_t period_us, uint32_t match_value, uint8_t mode)
165 {
166     //Configuración de Timer en Modo Temporizador
167     TIM_TIMERCFG_Type  cfgTimerMode;
168     //Configuración del Tiempo de Desborde de MATCH
169     TIM_MATCHCFG_Type  cfgTimerMatch;
170
171     //Base de Tiempo del Prescaler a partir de valor en µs
172     cfgTimerMode.PrescaleOption = TIM_PRESCALE_USVAL;
173     //Base de Tiempo de Desborde de Prescaler [µs] (Time_Overflow_PC)
174     cfgTimerMode.PrescaleValue  = period_us;
175
176     //Habilitación de Canal de MATCH1: (MR1)
177     cfgTimerMatch.MatchChannel  = 1;
178     //Valor Máximo para TC (MR)
179     cfgTimerMatch.MatchValue    = match_value;
180     //Habilitación de Interrupción por MATCH: (MR1I=0)
181     cfgTimerMatch.IntOnMatch    = DISABLE;
182     //Habilitación de Reset por MATCH: (MR1R=1)
183     cfgTimerMatch.ResetOnMatch  = ENABLE;
184     //Deshabilitación de Stop por MATCH: (MR1S=0)
185     cfgTimerMatch.StopOnMatch   = DISABLE;
186     //Modalidad TOGGLE en P1.29
187     cfgTimerMatch.ExtMatchOutputType = mode;
188
189     //Configuración del Modalidad del TIMER0
190     TIM_Init(LPC_TIM0, TIM_TIMER_MODE, &cfgTimerMode);
191     //Configuración del TIMER0 como Temporizador
192     TIM_ConfigMatch(LPC_TIM0, &cfgTimerMatch);
193
194     //Deshabilitación de Interrupción por TIMER0
195     NVIC_DisableIRQ(TIMER0_IRQn);
196
197     //Habilitación del TIMER0
198     TIM_Cmd(LPC_TIM0, ENABLE);
199
200     return;
201 }
202
203 /*****
204  * @brief   Configura e inicializa el periférico ADC.
205  *
206  * @details Configura la conversión del canal AD0.0 disparada por flanco
207  *           descendente en el hardware MAT0.1 del Timer 0.

```

```

208 *
209 * @param <rate> Tasa de conversión del ADC [S/s]
210 *
211 * @return None
212 *****/
213 void cfgADC(uint32_t rate)
214 {
215     // Inicialización del ADC con Tasa de Conversión [S/s]
216     ADC_Init(LPC_ADC, rate);
217     // Deshabilitación del Modo Burst para ADC: (BURST=0)
218     ADC_BurstCmd(LPC_ADC, DISABLE);
219     // Inicialización de Conversión debido a MAT0.1: (START=100)
220     ADC_StartCmd(LPC_ADC, ADC_START_ON_MAT01);
221     // Inicialización de Conversión ante Flanco Descendente debido a MAT0.1: (EDGE=1)
222     ADC_EdgeStartConfig(LPC_ADC, ADC_START_ON_FALLING);
223     // Funcionalidad de P0.23 como AD0.0 (Canal 0)
224     ADC_ChannelCmd(LPC_ADC, ADC_CHANNEL_0, ENABLE);
225     // Habilitación de Interrupción ante Finalización de Conversión por AD0.0
226     ADC_IntConfig(LPC_ADC, ADC_ADINTEN0, ENABLE);
227
228     // Habilitación de Interrupción por ADC0
229     NVIC_EnableIRQ(ADC_IRQn);
230
231     return;
232 }
233
234 /*****
235 * @brief Rutina de Servicio de Interrupción (ISR) para el vector ADC.
236 *
237 * @details Implementa una secuencia de adquisición de una etapa:
238 * 1. Lee el canal ADC asociado al sensor.
239 * 2. Convierte el resultado digital a unidades eléctricas.
240 * 3. Convierte la magnitud eléctrica a unidades físicas.
241 * 4. Procesa la variable adquirida y actualiza la alarma.
242 *
243 * @param None.
244 * @return None.
245 *****/
246 void ADC_IRQHandler(void)
247 {
248     // Espera Fin de Conversión de AD0.0: DONE_CH0 = 1?
249     if (ADC_ChannelGetStatus(LPC_ADC, LM35_SENSOR_CHANNEL, ADC_DATA_DONE) == SET){
250
251         // Guarda Dato Digitalizado de AD0.0 (12 bits) en Cuentas de AD
252         lm35_raw = ADC_ChannelGetData(LPC_ADC, LM35_SENSOR_CHANNEL);
253
254         // Conversión de Cuentas AD a Tensión de Entrada [mV]
255         lm35_mv = (uint32_t)(lm35_raw * ADC_VREF_MV) / ADC_MAX_COUNTS;
256
257         // Conversión de Tensión de Entrada a Temperatura [°C]
258         lm35_temp_c = lm35_mv / LM35_SENSITIVITY_MV;
259
260         // Procesamiento del Dato Acondicionado en Unidad Física [°C]
261         if (lm35_temp_c > ALARM_TEMP_C) {
262             GPIO_SetValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
263         } else {
264             GPIO_ClearValue(ALARM_HIGH_PORT, ALARM_HIGH_PIN_MASK);
265         }
266
267         return;
268     }

```